Adaptive Fairness Constraints for Online Learning Under Concept Drift

Tiago Silva
Faculty of Science and Technology
University of Coimbra
Coimbra, Portugal
tds@student.uc.pt

Bruno Veloso
INESC-TEC
Porto, Portugal
Bruno.m.veloso@inesctec.pt

João Gama
INESC-TEC
Porto, Portugal
joao.gama@inesctec.pt

Pedro Henriques Abreu
Faculty of Science and Technology
University of Coimbra
Coimbra, Portugal
pha@dei.uc.pt

***Abstract*—Online learning systems in nonstationary environments must stay both predictive and fair as the data distribution evolves. Existing fairness-aware learners optimize a fixed regularizer ignoring concept drift, while drift-adaptive streaming methods enforce no fairness. We propose Fairness-Aware Drift-Aware Online Learning (FADO), which extends a fair oblique decision forest with an adaptive fairness constraint and a drift reaction controller. Instead of weighting the fairness penalty by a constant chosen in advance, FADO bounds demographic parity and treats the penalty weight as the Lagrange multiplier of that bound, tracked online by stochastic dual ascent on a per-sample Horvitz–Thompson estimate of each group's disparity that we prove to be unbiased. A dual-threshold ADWIN detector and a fairness monitor reset the penalty's running statistics on confirmed drift, and the reaction controller raises the learning rate and sharpens the forest's routing, annealing both back during recovery. Over 30 seeds, on a natural cross-year shift in census income data and on four types of concept drift injected into the Adult stream, FADO lowers demographic parity by 25 to 33 percent relative to its base learner in every setting, significantly below all three published baselines over the whole stream and after the drift, at a whole-stream cost of at most 0.02 in F1. Against its base learner given tuning, in hindsight, the constant weight for the one that FADO, on average, reaches online, it is significantly better in 12 and worse in 7 of 27 comparisons, and better on every metric after a feature-permutation drift.**

***Index Terms*—fairness-aware online learning, concept drift, constrained learning, drift detection, group fairness, oblique decision forests**

I. Introduction

The growing adoption of machine learning (ML) systems in critical decision-making domains, including healthcare [1], criminal justice [2], and hiring [3], where their predictions may directly affect individuals and society, makes optimizing for performance an insufficient objective; models must also satisfy fairness requirements to ensure equitable treatment across individuals or groups of individuals [4]. In this regard, among the fairness literature, *group fairness* [5] has emerged as one of the most widely studied paradigms, aiming to ensure that predictive behavior is balanced across populations defined by sensitive attributes such as gender, race, or age. Common formulations include demographic parity (DP), equal opportunity, and equalized odds (EO), each imposing statistical constraints on model outcomes across groups (sometimes mutually incompatible) [5], [6]. Most existing fairness-aware models, unfortunately, are developed under the assumption of a stationary data distribution. In practice, real-world environments are inherently dynamic, and properties of data evolve over time due to policy changes, behavioral shifts, seasonal effects, economic fluctuations, or other social transformations. This phenomenon, commonly referred to as concept drift, occurs when the relationship between input variables and target outcomes changes over time, causing previously learned decision boundaries to become outdated. Different forms of concept drift [7] require different adaptation strategies and can degrade performance when models fail to adjust. Drift also threatens fairness: shifting distributions alter the statistical relationships that support fairness guarantees, increasing bias against some groups. To address this challenge, the online learning and data stream mining literature has proposed numerous adaptive methods capable of detecting and responding to concept drift in real time. These approaches often rely on drift detection mechanisms combined with incremental learning strategies to maintain predictive accuracy under nonstationary conditions. However, the majority of existing methods focus primarily on predictive utility [8], [9], while fairness considerations remain comparatively underexplored. As a result, fairness guarantees established during training may deteriorate over time even when predictive accuracy remains relatively stable [10]. This limitation is particularly critical in high-stakes applications, where persistent fairness violations may remain undetected for extended periods, compromising trustworthiness, reinforcing societal inequalities, and causing tangible harm to affected groups. These limitations highlight an important gap in the literature and motivate the following research questions (RQs):

- **RQ1:** To what extent does concept drift affect group

fairness metrics in online learning settings?
- **RQ2:** Can adaptive drift detection and response mechanisms preserve fairness guarantees while maintaining predictive performance over time?

Based on these research questions, this work investigates fairness-aware online learning under concept drift. We introduce **F**airness-**A**ware **D**rift-Aware **O**nline Learning (FADO), a framework that leverages an existing soft-routed fair oblique decision forest, Aranyani [11], extending it with three mechanisms: an *adaptive fairness constraint*, which replaces the fixed fairness weight of the base learner with the Lagrange multiplier of a bound on the fairness metric and tracks it online; a dual-threshold drift detector, complemented by a fairness monitor, whose confirmed drifts reset the fairness penalty's running statistics; and a reaction controller that modulates the optimizer learning rate and a routing temperature, which FADO adds to the forest, in response to detected change. We emphasize that ARANYANI is not contributed by this paper: it serves as the in-processing base learner around which FADO builds its constraint, drift-detection, and response machinery. Our central hypothesis is that fairness degradation under concept drift can be mitigated by enforcing fairness as a constraint whose price adapts to the data, coupled with explicit drift detection and response, allowing the model to maintain both predictive utility and equitable treatment across groups in nonstationary environments.

To evaluate this hypothesis, we run our evaluation on two complementary testbeds. FOLKTABLES (American Community Survey (ACS) Income, California) contributes a natural cross-year shift between 2015 (training) and 2017–2018 (prequential test), and ADULT serves as a controlled testbed into whose stream we inject four types of concept drift [12], one per scenario. FADO is compared against three streaming baselines spanning the accuracy/fairness/adaptivity design space: the Adaptive Random Forest (ARF) [8], an accuracy-oriented ensemble with drift detection but no fairness mechanism; robust fairness regularization (RFR) [13], a sharpness-aware fairness-reweighted method without adaptive drift handling; and a controller-free ARANYANI baseline, identical to the base learner inside FADO but with a fixed fairness weight and no drift handling, which isolates FADO's contribution. To separate the benefit of adapting the weight from that of a larger one, we also give this baseline, in hindsight, the constant weight that FADO reaches online. The main contributions of this work are:

- FADO, a fairness-aware online learning framework that wraps a fair oblique decision forest with an adaptive fairness constraint, a dual-threshold ADWIN drift detector with a fairness monitor, and a reaction controller acting on the learning rate and on a routing temperature that FADO introduces;
- Where ARANYANI relaxes its node-level fairness constraints into a penalty with a fixed weight, a formulation that keeps a bound on the demographic parity of the predictions and treats the penalty weight as its dual price, tracked online, together with a per-sample Horvitz–Thompson estimate of each group's disparity that we prove unbiased and that drives both the price and the fairness monitor;
- An empirical analysis over 30 seeds, on a natural cross-year shift on FOLKTABLES and on four injected drift types on ADULT, of how concept drift affects group fairness over the stream, comparing FADO against the ARF, RFR, and controller-free ARANYANI baselines, and against ARANYANI given FADO's weight in hindsight, with paired, multiplicity-corrected tests, together with an analysis of how often and how fast FADO detects each drift; code, per-seed results, and the scripts that generate every table and number are released.

II. RELATED WORK

Following the most adopted taxonomy [14], in this section, the works will be divided into three categories: (i) pre-processing methods, (ii) in-processing methods, and (iii) post-processing methods.

A. Pre-Processing Methods

Pre-processing methods aim to mitigate bias in the data before the training stage of a model. Xiong et al. [15] propose leveraging training samples reweighting to satisfy demographic parity while minimizing distributional distortion using the Wasserstein distance to the original dataset. Although effective, this method does not account for concept drift, nor does it prove robustness to other forms of fairness violations, which limits its applicability in dynamic environments. Zeng et al. [16] propose a different approach that contrasts with the aforementioned, modifying the actual data labels by treating the removal of discrimination from the training dataset as a randomized response problem. The main contribution of this work is the provision of a theoretical bridge between pre-processing techniques and downstream Bayes-optimal classifiers, and the fact that it is model agnostic: because it only alters the training labels, users do not need to modify their current pipelines. However, this method also does not account for concept drift or its impact, and it is limited by the binary setup, meaning it only works with binary sensitive attributes and binary outcomes.

B. In-Processing Methods

In-processing methods incorporate fairness constraints directly into the learning algorithm. Four lines are especially relevant to our work.

Chowdhury et al. [11] propose ARANYANI, an online fair-learning ensemble of soft-routed oblique decision trees in which the demographic-parity penalty is enforced at the level of individual routing nodes, inducing a parameter-isolation property that keeps the per-step fairness gradient cheap and lets the ensemble track running per-group statistics rather than storing past samples. ARANYANI starts from node-level fairness constraints but relaxes them into a regularizer whose weight is a fixed hyperparameter, and it has no mechanism for

drift. We adopt it as our base learner; these two limitations motivate our extension.

Gomes et al. [8] propose ARF, a streaming ensemble of Hoeffding trees with per-tree warning and drift detectors that swap in background trees on confirmed drift. ARF is the standard streaming-adaptation baseline but is purely accuracy-oriented; we treat it as a representative adaptive-but-not-fair baseline.

Jiang et al. [13] model distribution shift as a bounded weight-space perturbation and optimize for worst-case fairness under that perturbation, producing the sharpness-aware reweighting (RFR) that we use as our fair-but-not-adaptive baseline.

Finally, fairness can be imposed as a constraint rather than as a weighted penalty. Zafar et al. [17] bound a covariance proxy of disparate impact, Agarwal et al. [18] reduce fair classification to a saddle-point problem solved by a sequence of cost-sensitive learners, and Cotter et al. [19] optimize non-differentiable rate constraints with a proxy-Lagrangian game. These methods solve the problem offline on a fixed training set; online convex optimization with long-term constraints [20] provides the primal–dual template that FADO follows one sample at a time.

C. Post-Processing Methods

Post-processing approaches transform an already-trained classifier's outputs. Hardt et al. [6] introduce randomized threshold adjustments that equalize group error rates and remain the standard model-agnostic baseline; Mishler et al. [21] extend the idea to counterfactual equalized odds in intervention settings (at the cost of strong causal assumptions); Small et al. [22] replace fixed randomization regions with smooth monotonic probability curves to recover individual fairness. None of these methods touch the training loop, so none of them adapt under drift; we mention them for completeness rather than as direct comparators.

D. Main Limitations of Prior Work

Notwithstanding the significant contributions of the families surveyed above, we identify four main limitations that jointly motivate this work and the design of FADO:

- **L1 – Static fairness assumption.** Once the protected-attribute conditional or the label distribution shifts, the fairness guarantees obtained at training time provide no protection at deployment;
- **L2 – Adaptive methods are accuracy-oriented.** Streaming ensembles such as ARF [8] explicitly detect and react to drift but do so purely on the predictive-error signal, with no mechanism for tracking or correcting fairness violations;
- **L3 – Existing fair online learners do not react to drift.** ARANYANI [11] provides memory-efficient online fairness, but its regularizer is nonadaptive: when the distribution moves it continues to optimize the same penalty with the same fixed weight, although the weight that keeps disparity acceptable changes with the data, and convergence to the new optimum is slow;
- **L4 – Drift-aware fair methods rely on assumptions that do not transfer to streams.** Approaches such as [13] treat distribution shift as a bounded weight-space perturbation, which is useful in worst-case analysis but does not yield an actionable online detector or controller, and constrained fair learners [17]–[19] solve their problem offline.

FADO addresses L1–L4 directly by retaining the node-level fairness regularizer of ARANYANI as the in-processing base learner and adding on top of it (i) an adaptive fairness constraint, whose dual price replaces the fixed fairness weight; (ii) an ADWIN-based dual-threshold detector on the online error stream and a fairness monitor on a per-sample disparity signal, whose confirmed drifts reset the regularizer's running statistics; and (iii) a reaction controller that modulates the optimizer learning rate and a routing temperature added to the forest on warning and confirmation events.

III. PROBLEM STATEMENT

A. Online Learning Setting

We consider an online binary classification problem over a data stream $\{(x_t, y_t, a_t)\}_{t=1}^{T}$, where $x_t \in \mathbb{R}^d$ is the feature vector observed at time t, $y_t \in \{0, 1\}$ is the binary target label, and $a_t \in \mathcal{A}$ is a sensitive attribute taking values in a finite set with $|\mathcal{A}| = K \geq 2$. Following the formulation of previous work [11], [23], at each time step the learner receives x_t, produces a prediction $\hat{y}_t = \arg\max f_t(x_t)$, then observes (y_t, a_t) and updates its parameters under a prequential, test-then-train protocol. The fairness formulation introduced below supports arbitrary K; in our study each dataset has one binary protected attribute (sex), so $K = 2$.

B. Group Fairness Metrics

We focus on *group fairness*, which assesses whether model behavior is statistically similar across protected groups. ARANYANI [11] defines demographic parity on the forest's outputs for two groups and extends its formulation to more groups and to equalized odds in its appendix; we measure fairness on the *predictions*, in a form that holds for any number of groups. Let $\mathcal{A}_t \subseteq \mathcal{A}$ denote the set of sensitive groups observed within the current evaluation window, and let $\hat{p}_a = \mathbb{E}[\hat{y} \mid a]$ be the positive-prediction rate for group $a \in \mathcal{A}_t$. We define the unweighted mean group rate as:

$$\bar{p} = \frac{1}{|\mathcal{A}_t|} \sum_{a' \in \mathcal{A}_t} \hat{p}_{a'}, \tag{1}$$

i.e., the average positive-prediction rate across the observed groups. The *demographic parity violation* is then the worst-case deviation of any group's positive-prediction rate from this mean:

$$D_a = \bar{p} - \hat{p}_a, \qquad \mathrm{DP} = \max_{a \in \mathcal{A}_t} |D_a|, \tag{2}$$

which is bounded between 0 (perfect parity) and 1. DP measures bias in the prediction marginal and is sensitive to

differences in base rates across groups. We adopt this mean-from-average form rather than the more common pairwise gap $\max_{a,a'} |\hat{p}_a - \hat{p}_{a'}|$ because it mirrors the node-level regularizer as we implement it (Section IV-A). For a binary attribute ($K = 2$) this form reduces to $\mathrm{DP} = \frac{1}{2}|\hat{p}_0 - \hat{p}_1|$, equivalent up to a factor of two to the textbook pairwise gap.

We also report *equalized odds*, the same deviation measured within each true-label class. With $\hat{p}_{a|y} = \mathbb{E}[\hat{y} \mid a, y]$ and $\bar{p}_y$ its mean across the observed groups:

$$\mathrm{EO} = \max_{y\in\{0,1\}} \max_{a\in\mathcal{A}_t} \left|\hat{p}_{a|y} - \bar{p}_y\right|. \tag{3}$$

Restricting it to $y{=}1$ gives equal opportunity. EO separates bias from differences in base rates. We report it but do not optimize it: ARANYANI derives the label-conditional node regularizer [11, App. C.1], and constraining EO in FADO would also need a label-conditional version of the per-sample signal (10).

C. From a Fixed Penalty to a Fairness Constraint

ARANYANI [11] states fairness as constraints on the forest's internal routing nodes, but then relaxes them into a regularized objective $\mathcal{L}_{\mathrm{CE}} + \lambda\,\mathcal{L}_{\text{fair}}$ (Section IV-A) whose weight λ is fixed in advance, a knob with no meaning outside the dataset on which it was tuned. What a deployment can state and defend instead is a tolerance: the model's demographic parity must not exceed ε. We therefore pose:

$$\min_{\theta}\ \mathbb{E}\big[\mathcal{L}_{\mathrm{CE}}(\theta)\big] \quad \text{s.t.} \quad \mathrm{DP}(\theta) \le \varepsilon, \tag{4}$$

and ask the learner to satisfy it *at every point of a drifting stream*. Under drift the multiplier that makes (4) bind changes over time: the weight that keeps disparity acceptable before a shift is too small after a shift that widens the gap between groups, and needlessly costly when the stream is already fair, so it must be estimated online rather than tuned once.

IV. METHODOLOGY AND EXPERIMENTAL DESIGN

FADO builds on the ARANYANI fair oblique forest [11], which we use as the base learner and summarize only as far as FADO relies on it (Section IV-A). On top of it, FADO adds three mechanisms (Fig. 1): (i) an adaptive fairness constraint, whose dual price replaces the base learner's fixed fairness weight (Section IV-B); (ii) drift detection with forgetting, in which a dual-threshold ADWIN detector on the error stream and a fairness monitor on a per-sample disparity signal reset the fairness statistics on confirmed drift; and (iii) a reaction controller on the learning rate and on a routing temperature that FADO adds to the forest (both in Section IV-C). Sections IV-D–IV-I then describe datasets, drift scenarios, baselines, hyperparameter selection, the evaluation protocol, and reproducibility.

A. Base Learner

We use ARANYANI [11] as the base learner and refer to [11] for its details. It is an ensemble of M soft oblique trees of depth h whose internal node j routes an input to its left child with probability $n_j(x) = \sigma(w_j^\top x + b_j)$ and to its right child otherwise. Fairness is imposed on these routing decisions: ARANYANI constrains the deviation $F_{j,a}$ of each node's average routing decision for group a from its average over all inputs (for two groups, the difference between the groups' averages), the per-node analogue of D_a in (2), and then relaxes the constraints into the regularized objective:

$$\mathcal{L} = \mathcal{L}_{\mathrm{CE}} + \lambda\,\mathcal{L}_{\text{fair}}, \qquad \mathcal{L}_{\text{fair}} = \sum_j \frac{1}{K} \sum_{a\in\mathcal{A}} \phi_\delta\big(F_{j,a}\big), \tag{5}$$

where $\mathcal{L}_{\mathrm{CE}}$ is the cross-entropy loss, ϕ_δ is a Huber surrogate, and λ is a fixed hyperparameter. It estimates the fairness gradient online from running per-group averages of the routing decisions and of their gradients, the statistics S, so no past sample is stored. Our implementation, used identically by FADO and the ARANYANI baseline, takes the cross-group mean unweighted and gives the linear branch of ϕ_δ a unit slope.

FADO changes this learner in three places. First, the fixed weight λ becomes the price λ_t of Section IV-B. Second, the statistics S are reset when a drift is confirmed. Third, FADO adds a routing temperature, $n_j(x) = \sigma\big((w_j^\top x + b_j)/\tau\big)$, which recovers ARANYANI at $\tau = 1$ and which the reaction controller lowers after a confirmed drift to make routing nearly hard (Section IV-C).

B. Adaptive Fairness Constraint

FADO solves (4) online by treating the weight λ in (5) as a Lagrange multiplier.

One-Sided Linear Constraints: Since $|x| \le \varepsilon$ holds exactly when $x \le \varepsilon$ and $-x \le \varepsilon$, the constraint $\mathrm{DP} \le \varepsilon$ is equivalent to the $2K$ one-sided constraints:

$$+D_a \le \varepsilon, \qquad -D_a \le \varepsilon, \qquad a \in \mathcal{A}. \tag{6}$$

The rewrite has no absolute value and no maximum, and it states *which side* of the mean a group falls on. Its purpose is statistical: each constraint is linear in D_a, so an unbiased estimate of D_a is an unbiased estimate of every constraint's value. An absolute value would not preserve this — $\mathbb{E}|v| \neq |\mathbb{E}v|$ — and a controller fed with the magnitude of a noisy disparity estimate would read noise as unfairness and raise the price indefinitely on a perfectly fair model.

Lagrangian and Dual Ascent: Attaching a multiplier $\mu_{a,\pm} \ge 0$ to each constraint gives the Lagrangian:

$$\mathcal{L}(\theta, \mu) = \mathcal{L}_{\mathrm{CE}}(\theta) + \sum_a \Big[\mu_{a,+}(D_a - \varepsilon) + \mu_{a,-}(-D_a - \varepsilon)\Big], \tag{7}$$

and (4) is the saddle point $\min_\theta \max_{\mu\ge 0} \mathcal{L}$ [24]. The gradient of $\mathcal{L}$ in $\mu_{a,\pm}$ is the violation $\pm D_a - \varepsilon$, so projected ascent reads:

$$\mu_{a,\pm} \leftarrow \Pi_{[0,\lambda_{\max}]}\big(\mu_{a,\pm} + \eta_\lambda(\pm D_a - \varepsilon)\big), \tag{8}$$

interleaved with the usual gradient step in θ (primal–dual descent–ascent [20]). In control terms, μ *integrates* the violation: it keeps rising while a group's deviation exceeds

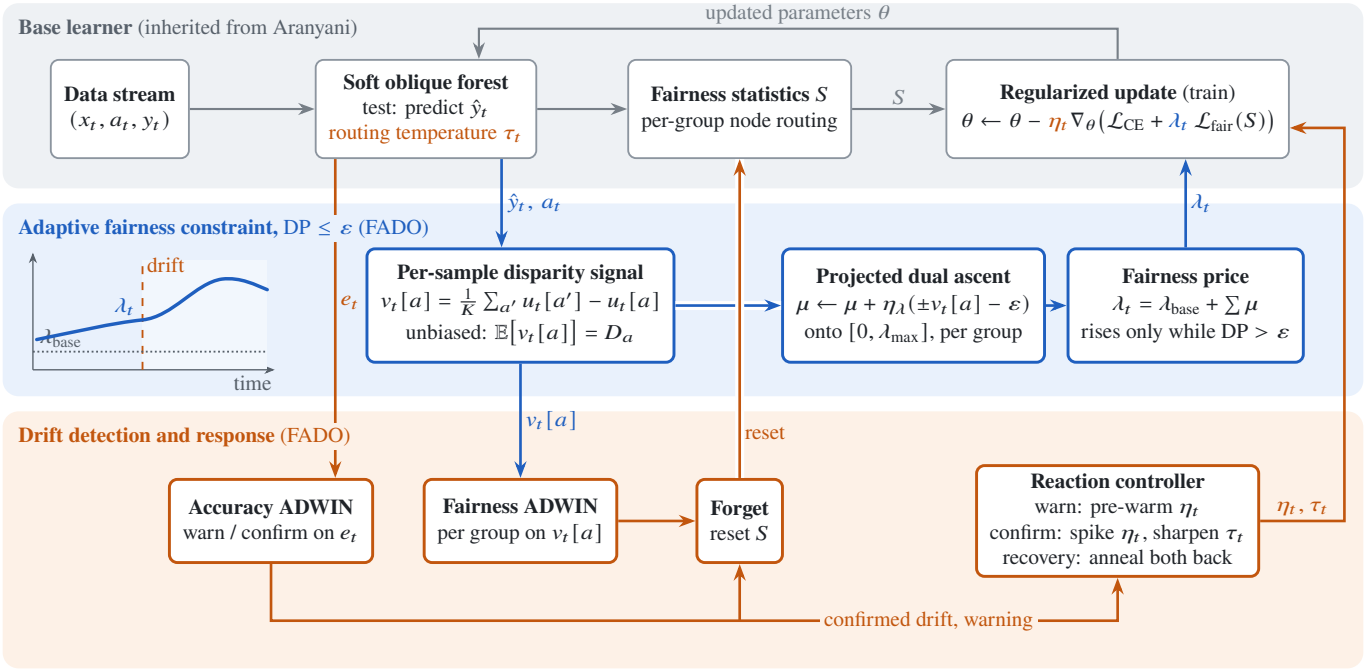


Fig. 1. Overview of FADO. Gray: the ARANYANI base learner [11], which predicts and then trains on each sample. Blue: the adaptive fairness constraint, which turns each prediction into a disparity signal and sets the fairness price λ_t. Orange: drift detection and response, which resets the fairness statistics S on drift and adjusts the learning rate η_t and routing temperature τ_t. Colored symbols in gray boxes are set by FADO.

ε and stops only when it does not, so it has no steady-state error. While a group's deviation stays below ε, its multiplier drains toward zero, so the penalty weight decreases gradually while the stream remains fair. The projection keeps the prices nonnegative, as duality requires, and bounded to protect accuracy. FADO sums the prices into a single weight for ARANYANI's node-level penalty:

$$\lambda_t = \min\Big(\lambda_{\text{base}} + \sum_{a,\pm} \mu_{a,\pm},\ \lambda_{\max}\Big), \tag{9}$$

where λ_{base} is the fixed weight used by the ARANYANI baseline. FADO therefore never applies less fairness pressure than ARANYANI, and adds pressure only while the constraint is violated. The floor is not needed by the method: with $\lambda_{\text{base}} = 0$, the weight is set entirely by the prices, that is, by the constraint. We use $\lambda_{\text{base}} > 0$ only to make the comparison with ARANYANI fair, since both models then start from the same weight and FADO's gain cannot come from a larger initial penalty. Penalizing the nodes is a sound way to reduce output disparity: ARANYANI shows that when every node treats the groups alike, the demographic parity of the forest's output is bounded [11, Lemma 1].

A Per-Sample Estimate of the Violation: Online, D_a is never observed: each step reveals one individual, from one group. We therefore build, from that single sample, a quantity that equals D_a in expectation. Let $\pi_a = \Pr(a_t = a)$ and define:

$$u_t[a] = \frac{\hat{y}_t\,\mathbf{1}[a_t = a]}{\pi_a}, \qquad v_t[a] = \frac{1}{K}\sum_{a'} u_t[a'] - u_t[a]. \tag{10}$$

Proposition 1 (Unbiased per-sample disparity): Conditional on the current model, if (x_t, a_t) is drawn from the stream distribution, then $\mathbb{E}[u_t[a]] = \hat{p}_a$ and $\mathbb{E}[v_t[a]] = D_a$ for every group a. Consequently $\pm v_t[a] - \varepsilon$ is an unbiased estimate of the violation of each constraint in (6).

Proof: $u_t[a]$ is nonzero only if $a_t = a$ and $\hat{y}_t = 1$, an event of probability $\pi_a \hat{p}_a$, in which case it equals $1/\pi_a$. Hence $\mathbb{E}[u_t[a]] = \hat{p}_a$, the Horvitz–Thompson identity [25]. By linearity, $\mathbb{E}[v_t[a]] = \frac{1}{K}\sum_{a'} \hat{p}_{a'} - \hat{p}_a = D_a$. The constraints in (6) are linear in D_a, which gives the last claim. ■

Substituting v_t for D_a in (8) yields a stochastic dual ascent whose steps are correct on average. The signal needs no window, so it responds from the first sample after a change, and successive values are conditionally independent, whereas a rolling disparity over W samples is almost perfectly auto-correlated. In practice π_a is replaced by the running share $\hat{\pi}_a$ (floored at 0.02), which makes (10) a consistent ratio estimator.

Remark 1 (Expected dynamics): Under a sustained violation $\Delta_{\text{DP}} = \text{DP} - \varepsilon > 0$, the active multipliers grow by about $\eta_\lambda \Delta_{\text{DP}}$ per sample; once DP $< \varepsilon$ they drain by at most $\eta_\lambda \varepsilon$ per sample, so draining is slow by design. With two groups, $D_0 = -D_1$, so the constraints in (6) coincide in pairs; our implementation keeps all four multipliers, which leaves the duality intact but makes the effective dual step $2\eta_\lambda$.

C. Drift Detection and Reaction Controller

The controller observes the online error stream $e_t = \mathbf{1}[\hat{y}_t \neq y_t]$ through two independent adaptive windowing (ADWIN) detectors [26]: a pre-warm detector D_w and a confirmation detector D_c, parameterized by significance levels δ_w and $\delta_c <$

δ_w (in ADWIN a larger δ is more sensitive, so the pre-warm detector fires first). Both are paired with a label-noise guard that excludes a sample from triggering an event whenever an incorrect prediction is emitted with post-softmax confidence above ν, preventing single confident-but-wrong samples from spuriously activating the controller. A minimum of m_0 samples per detector is enforced before the first activation to avoid early-stream false positives.

When D_w fires, the optimizer learning rate is multiplied by β_w, pre-warming the parameters for a potential change. When D_c subsequently confirms a drift, four actions are taken atomically: (i) the learning rate is set to $\beta_c \cdot \eta_0$; (ii) the routing temperature that FADO adds (Section IV-A) is set in every tree to τ_{drift}, sharpening routing decisions while the ensemble refits; (iii) the running fairness statistics S of Section IV-A are reset and rebuilt from post-drift samples; and (iv) both detectors are reset and a cooldown of C samples is enforced before a new event can fire. Recovery is signaled when the rolling accuracy returns to the pre-drift baseline (estimated over the window preceding the drift) or after a timeout of 2Δ samples, after which τ is annealed back toward its steady-state value at rate $\Delta\tau$ per step and η decays linearly to η_0 over Δ steps.

The two-stage design balances a confirmation-only reaction (too slow under sudden drift) against pre-warming without a gate (amplifies noise). The reset in (iii) matters for the adaptive constraint: the price of Section IV-B tracks the *current* disparity, but the regularizer's direction comes from statistics accumulated since the last reset, so without forgetting a correctly raised price would push along the pre-drift direction. For the same reason a *fairness monitor*: one ADWIN detector per group on the per-sample signal $v_t[a]$ of (10) also resets the statistics when a group's disparity shifts; it does not change the learning rate or the temperature. Because the signal is unbiased and not autocorrelated, this detector avoids the false alarms that a rolling-window disparity would cause. Concrete values for all controller parameters are reported in Table II. Fig. 2 combines the three mechanisms into FADO's prequential test-then-train loop.

Require: stream $\{(x_t, y_t, a_t)\}$; settings of Table II
init forest f, optimizer at η_0, temperature τ, multipliers $\mu \leftarrow 0$, running fairness stats S
for $t = 1$ **to** T **do**
  $\hat{y}_t \leftarrow \arg\max f(x_t)$; $e_t \leftarrow \mathbf{1}[\hat{y}_t \neq y_t]$ ▷ test
  update windowed accuracy, DP, and EO over the last W samples
  noise $\leftarrow e_t \wedge (\text{conf}(\hat{y}_t) > \nu)$ ▷ label-noise guard
  ok $\leftarrow \neg$*noise* $\wedge\ (t - t_{\text{last}} \geq C)$ ▷ guard, cooldown
  if $D_w(e_t)$ warns **and** *ok* **then** $\eta \leftarrow \beta_w\, \eta_0$ ▷ pre-warm
  end if
  if $D_c(e_t)$ confirms **and** *ok* **then**
    $\eta \leftarrow \beta_c\, \eta_0$; $\tau \leftarrow \tau_{\text{drift}}$; reset S; $t_{\text{last}} \leftarrow t$ ▷ spike + sharpen
  end if
  compute $v_t[a]$ by (10); reset S if a fairness detector confirms
  $\mu_{a,\pm} \leftarrow \Pi_{[0,\lambda_{\max}]}\big(\mu_{a,\pm} + \eta_\lambda(\pm v_t[a] - \varepsilon)\big)$ ▷ dual ascent
  $\lambda_t \leftarrow \min(\lambda_{\text{base}} + \sum \mu,\ \lambda_{\max})$
  if recovered **or** timed out **then** anneal $\eta \to \eta_0$ over Δ, $\tau \to 1$
  accumulate (x_t, y_t, a_t) into S ▷ train
  optimizer step on $\nabla\big(\mathcal{L}_{\text{CE}}(f(x_t), y_t) + \lambda_t\, \mathcal{L}_{\text{fair}}(S)\big)$ at rate η
end for

Fig. 2. The FADO prequential test-then-train loop.

TABLE I
DATASETS

Dataset	Size	Target	Protected attribute	Split / stream
FOLKTABLES (ACS Income, CA)	187,474 + 187,475	Income $>$ \$50k	Sex (2 groups)	Train 2015; test 2017–2018 in temporal order; both splits uniformly subsampled to 10% per seed
ADULT	32,561 + 16,281	Income $>$ \$50k	Sex (2 groups)	Standard train/test split; one injected drift per test stream

D. Datasets and Preprocessing

Table I summarizes the two datasets. FOLKTABLES (ACS Income, CA) [27] supplies a large natural cross-year shift between 2015 (training) and 2017–2018 (prequential test); ADULT [28] provides a controlled testbed on which we inject concept-drift perturbations of a known form on the test stream. Both datasets are processed through a shared tabular pipeline: numerical features are standardized using training-set means and standard deviations, and categorical variables are one-hot encoded. The encoder is fitted on the training split alone and reused unchanged for the test stream, so the streaming protocol remains free of test-time information leakage.

E. Concept-Drift Scenarios

We adopt complementary strategies on the two datasets. On FOLKTABLES, training on 2015 and evaluating prequentially on 2017–2018 already provides a real, multidimensional cross-year shift driven by genuine demographic and economic change; injecting additional synthetic perturbations on top of it would mask controlled and natural signals obscuring the source of any observed degradation.

On ADULT the test stream carries no known drift, so we inject it. We use four drift types [12], each giving a separate scenario: every model ran on all four, with one drift per stream at an onset drawn uniformly between 50% and 70% of the stream. The four types are *x_permutations*, where the feature vector is permuted by a fixed random permutation, changing the mapping from inputs to labels; *y_swaps*, where samples of one randomly chosen class are relabeled with probability 0.5, a concept drift in $P(y \mid x)$; *y_prior_skip*, where 75% of the samples of one class are dropped, a prior shift; and *x_exceed_skip*, where samples whose value of a randomly chosen numerical feature exceeds its training median are dropped, a covariate shift. The onset, the drifted class, the permutation, and the feature are drawn from the seed, so each seed is a different stream; the order of the ADULT test split is fixed.

F. Baselines

We compare FADO against three streaming baselines that span the accuracy/fairness/adaptivity design space, and against

an optimistic variant of the third:

- **ARF** [8]: the Adaptive Random Forest from the River library [29], used as an accuracy-oriented streaming reference that handles drift through per-tree warning and drift detectors and background trees, but without any fairness mechanism;
- **RFR**: a sharpness-aware reweighting variant that applies the RFR optimizer [13] (sharpness-aware minimization over a fairness-reweighted loss); this provides a fairness-aware static baseline without adaptive drift handling;
- **ARANYANI** [11]: the plain ARANYANI forest, with the fixed weight $\lambda = 0.1$ of its published configuration and no drift handling;
- **ARANYANI with leaked** λ: the same forest given a constant weight of at least FADO's mean λ_t on each dataset (0.5 on FOLKTABLES and 1.5 on ADULT), a value we read off FADO's own runs, and pre-trained with $\lambda_{\text{base}} = 0.1$ like FADO, so that both start from the same forest. A fixed-weight learner has no way to arrive at this value by itself: it could only obtain it by tuning λ separately for every dataset, whereas FADO reaches it online, without tuning. The leaked weight therefore gives ARANYANI an advantage it would not have in practice; we include this baseline to separate the benefit of a larger weight from the benefit of adapting it.

All methods are evaluated under the same test-then-train prequential protocol with batch size 1 as FADO so that comparisons are not biased by data ordering or data leakage. The global random number generators (NumPy, Python `random`, and TensorFlow) are pinned per seed and TensorFlow op determinism is enabled, so the data split, the forest initialization, and the training trajectory are identical across methods that share a seed.

G. Hyperparameter Selection

Table II lists the fairness and controller hyperparameters with their ADULT defaults and the FOLKTABLES overrides. For the base learner we keep the settings of [11]: $M = 3$ trees of depth $h = 4$, trained with Adam at $\eta_0 = 2{\times}10^{-3}$. The remaining values were set before the reported runs and were never tuned on the reporting seeds.

As argued in Section III-C, the fairness target $\varepsilon = 0.05$ is a fixed requirement rather than a tuned value: for two groups it allows a positive-prediction-rate gap of at most 0.10. The dual step $\eta_\lambda = 0.01$ trades responsiveness against noise in the per-sample signal (Remark 1), and the cap $\lambda_{\max} = 10$ bounds the accuracy that the price can spend when ε is hard to reach; the cap is not required by the method. The surrogate transition is $\delta = 0.1$, larger than the 0.01 used in [11].

The two ADWIN levels make the warning detector fire first: in ADWIN a larger δ is more sensitive, so $\delta_w = 2{\times}10^{-2}$ gives an early warning, while $\delta_c = 10^{-5}$ requires a strong, sustained rise of the error rate before the full reaction starts. The learning-rate multipliers β_w=5 and β_c=10 keep a 1:2 ratio: a warning gives a measurable pre-warm without committing the model to a new regime, and only a confirmed drift triggers the full spike. The drift temperature $\tau_{\text{drift}} = 0.1$, ten times lower than the steady-state $\tau = 1$, makes routing nearly hard during the spike, so the forest can move to a new leaf assignment within a few dozen samples instead of waiting for the soft routing weights to average over the new regime. The anneal $\Delta\tau = 2{\times}10^{-3}$ returns τ from 0.1 to 1 in 450 steps, well inside the $\Delta = 3000$ learning-rate decay horizon. The cooldown $C = 200$ stops the controller from firing again during its own response, and the label-noise guard $\nu = 0.7$ keeps wrong predictions from triggering an event. Because FOLKTABLES's drift is a gradual change between survey years rather than an abrupt break, we fixed a milder reaction for it in advance: a smaller spike ($\beta_c = 3$), a softer temperature ($\tau_{\text{drift}} = 0.5$), and a shorter horizon ($\Delta = 1000$), which avoid overreacting to a slow change.

TABLE II
FADO HYPERPARAMETERS

Symbol	ADULT	FOLKTABLES[a]	Role
Fairness regularizer and constraint			
λ_{base}	0.1	–	floor of λ_t; ARANYANI's fixed λ
ε	0.05	–	demographic-parity target
η_λ	0.01	–	dual step size
$\lambda_{\max}$	10	–	cap on λ_t
δ	0.1	–	quadratic/linear transition in ϕ_δ
W	1000	–	rolling window for reported metrics
Drift detection and reaction controller			
δ_w	2×10^{-2}	–	ADWIN significance (pre-warm)
δ_c	1×10^{-5}	–	ADWIN significance (confirmation)
δ_{fair}	2×10^{-3}	–	ADWIN significance (fairness monitor)
β_w	5	–	learning-rate multiplier on warning
β_c	10	3	learning-rate multiplier on confirmed drift
Δ	3000	1000	learning-rate decay horizon back to η_0
τ	1.0	–	steady-state routing temperature (added by FADO)
τ_{drift}	0.1	0.5	temperature set on confirmed drift
$\Delta\tau$	2×10^{-3}	–	per-step temperature anneal back to τ
C	200	–	cooldown (samples) between events
ν	0.7	–	label-noise confidence guard
m_0	30	–	minimum samples before first activation

[a] A dash means the FOLKTABLES value equals the ADULT value.

H. Evaluation Protocol

All methods are evaluated under a strict prequential test-then-train protocol with batch size 1 [30]: at each step the model emits $\hat{y}_t$, the prequential metrics are updated, and only then is the parameter update performed using the just-observed (y_t, a_t). We report mean and standard deviation over 30 reporting seeds, disjoint from the seeds used during development. The primary metrics are the F1 score of the positive class and the demographic-parity violation, with equalized odds

as a secondary, unoptimized criterion. Each is reported over the whole stream and over the part of the stream after the injected onset: DP and EO are computed over the same W-sample rolling window and averaged over that part, and F1 is computed from all its predictions.

All experiments ran on CPU, in Python 3.11 or 3.12, on three machines: a Linux workstation with an Intel Core i9-9900K, a Windows desktop with an Intel Core i5-8500, and an Apple M5 laptop. Each run executes all the methods of its sweep for one seed on one machine, so every paired comparison below is made on the same stream. A seed's stream is the same on every machine, and so are the results of FADO and of both ARANYANI variants: before the onset, the four ADULT scenarios of a seed share the same stream, and these methods' per-seed metrics on it are identical across scenarios that ran on different machines. ARF and RFR are reproducible on each machine but differ slightly across machines (a seed's demographic parity before the onset by at most 0.010).

For each metric column, paired-by-seed comparisons of FADO against each other method are performed using the two-sided Wilcoxon signed-rank test; p-values are corrected within each column of each block using the Holm–Bonferroni step-down procedure across the four methods compared with FADO (Section IV-F). As a stricter check, we also apply the correction across all the comparisons of Table III at once. The procedure follows the recommendations of Demšar [31]. We prefer the paired Wilcoxon test to the Welch two-sample t-test because it assumes no distribution for the per-seed metric and uses the pairing given by the shared data ordering (Section IV-F), which Welch ignores. The Welch results are reported alongside in the repository (Section IV-I).

I. Reproducibility

The full implementation, configuration files, seed manifests, and per-seed result files are released at https://github.com/TiagoSilva35/JustFADO. The repository lists the sweep configuration behind every reported run; one script regenerates Tables III and IV and the full significance report (paired Wilcoxon and Welch tests, raw and Holm-corrected p-values) from the per-seed results, and a second script recomputes, with its derivation, every number quoted in the text.

V. Results and Discussion

Table III reports F1, demographic parity (DP), and equalized odds (EO) as mean $\pm$ standard deviation over the 30 seeds, which all methods share, both over the whole stream and over the part of each ADULT stream that follows the injected onset; FOLKTABLES has no onset because its shift lies between the training year and the test stream. We measure utility with the F1 score of the positive class (income above \$50k) because that class is a minority: 40% of the FOLKTABLES stream and 24% of the ADULT test split. Bold marks the best mean of each column within a block. The column λ gives the fairness weight over the whole stream: for FADO, the mean of λ_t; for the two ARANYANI variants, their constant weight.

Every value of the other methods in Table III carries the outcome of its paired test against FADO (Section IV-H): $^\bullet$ if FADO is significantly better, $^\circ$ if it is significantly worse, and no mark if no significant difference is detected, which does not establish that the two are equivalent. The top of the table sums these outcomes over the 27 cells of each method: 3 metrics in 5 blocks over the whole stream and in 4 after the onset. Over the 108 comparisons, FADO obtains 79 significant wins and 15 significant losses; 11 of the losses are on F1, and the 4 on DP or EO are all against ARANYANI with leaked λ. With the Holm correction applied across all 108 comparisons at once, 81 of the 94 significant differences remain significant.

Table IV reports how often, and how fast, FADO's detectors register each injected drift. An onset counts as detected when a detector raises an alarm within $1{,}000$ samples after it, the length of one rolling window: a window-based detector cannot register a change before its window has refilled, so a later alarm is no longer attributed to the injected drift. The delay is the mean number of samples from the onset to that alarm over the detected runs only, so it must be read together with the detection rate; alarms not matched to an onset count as false alarms.

A. To What Extent Does Concept Drift Affect Group Fairness?

Not every type of concept drift affects group fairness the same way: the effect depends on the type of drift and on the fairness criterion. We measure it on ADULT, where the onset is known, by comparing each method's metrics before and after the onset on the same seeds. For the controller-free ARANYANI, the drifts that alter the relation between inputs and labels widen the gap in error rates between groups: equalized odds rises from 0.047 to 0.115 after the onset of *x_permutations* (an increase on all 30 seeds) and from 0.047 to 0.076 under *y_prior_skip*, where demographic parity also rises, from 0.082 to 0.097. Demographic parity, in contrast, falls after the onset of *y_swaps* (0.082 to 0.054), although equalized odds rises there too (0.047 to 0.069), and after the onset of *x_exceed_skip* (0.082 to 0.061): relabeling one class or removing the samples above a feature's median also brings the groups' positive-prediction rates closer. For ARF and RFR as well, demographic parity falls after the onset of *y_swaps* and *x_exceed_skip*, and it remains higher than FADO's before and after every onset. A fairness guarantee on the stream before a drift is, therefore, not preserved after it and the metric that worsens can't be known in advance. This is the setting the adaptive constraint targets.

On FOLKTABLES the shift lies between the training year and the whole test stream, so the stream offers no reference before the drift; every method shows a slightly lower demographic parity on the 2018 half of the stream than on the 2017 half, and we use FOLKTABLES to assess the methods under a natural, gradual shift rather than to measure the effect of drift itself.

TABLE III
F1, DEMOGRAPHIC PARITY, AND EQUALIZED ODDS OVER 30 SEEDS

Overall: FADO against each method over its 27 cells below	Significant wins	No significant difference	Significant losses
vs. ARANYANI	17	5	5
vs. ARANYANI, leaked λ	12	8	7
vs. ARF	26	1	0
vs. RFR	24	0	3
Total (108 comparisons)	79	14	15

	Whole stream				After the onset		
Method	F1↑	DP↓	EO↓	λ	F1↑	DP↓	EO↓
FOLKTABLES							
FADO	0.758±0.005	0.042±0.003	0.025±0.003	0.45			
ARANYANI	**0.762±0.005**°	0.057±0.004•	0.036±0.003•	0.1			
ARANYANI, leaked λ	0.759±0.005	**0.039±0.004**°	**0.024±0.003**°	0.5	*not applicable: no injected onset*		
ARF	0.640±0.009•	0.072±0.012•	0.077±0.016•	–			
RFR	0.729±0.005•	0.075±0.006•	0.059±0.006•	–			
x_permutations							
FADO	0.612±0.010	**0.060±0.004**	**0.053±0.009**	1.47	0.550±0.029	**0.062±0.010**	**0.064±0.022**
ARANYANI	0.609±0.019	0.083±0.007•	0.073±0.010•	0.1	0.513±0.056•	0.085±0.014•	0.115±0.025•
ARANYANI, leaked λ	0.590±0.021•	0.062±0.009	0.070±0.013•	1.5	0.493±0.063•	0.077±0.018•	0.100±0.028•
ARF	0.571±0.011•	0.090±0.003•	0.110±0.013•	–	0.519±0.026•	0.086±0.006•	0.119±0.026•
RFR	**0.626±0.006**°	0.100±0.004•	0.091±0.010•	–	**0.613±0.010**°	0.102±0.007•	0.104±0.019•
y_swaps							
FADO	0.645±0.058	0.048±0.004	**0.048±0.006**	0.94	0.526±0.222	**0.027±0.011**	**0.052±0.013**
ARANYANI	**0.647±0.047**	0.071±0.010•	0.056±0.010•	0.1	0.524±0.206	0.054±0.028•	0.069±0.024•
ARANYANI, leaked λ	0.636±0.054•	**0.044±0.007**°	0.055±0.011•	1.5	0.522±0.206	0.029±0.015	0.060±0.015•
ARF	0.615±0.063•	0.084±0.005•	0.096±0.011•	–	**0.561±0.164**	0.072±0.014•	0.083±0.017•
RFR	0.635±0.063•	0.089±0.015•	0.084±0.015•	–	0.504±0.245•	0.072±0.039•	0.083±0.037•
y_prior_skip							
FADO	0.676±0.051	0.059±0.011	**0.049±0.008**	1.41	0.621±0.229	**0.063±0.042**	**0.054±0.023**
ARANYANI	**0.695±0.049**°	0.083±0.017•	0.054±0.006•	0.1	**0.667±0.201**°	0.097±0.066•	0.076±0.018•
ARANYANI, leaked λ	0.679±0.053°	**0.058±0.015**	0.056±0.011•	1.5	0.656±0.206°	0.079±0.057•	0.067±0.025•
ARF	0.631±0.059•	0.091±0.022•	0.093±0.019•	–	0.522±0.305•	0.106±0.078•	0.089±0.044•
RFR	0.671±0.058•	0.099±0.021•	0.084±0.012•	–	0.629±0.240°	0.112±0.076•	0.092±0.031•
x_exceed_skip							
FADO	0.633±0.016	0.057±0.005	0.048±0.007	1.08	0.576±0.080	**0.049±0.013**	0.051±0.018
ARANYANI	**0.647±0.016**°	0.076±0.007•	**0.047±0.006**	0.1	**0.594±0.077**°	0.061±0.017•	**0.047±0.015**
ARANYANI, leaked λ	0.631±0.018•	**0.053±0.008**°	0.052±0.013	1.5	0.582±0.080°	0.050±0.017	0.051±0.024
ARF	0.586±0.022•	0.083±0.006•	0.090±0.013•	–	0.502±0.122•	0.065±0.020•	0.076±0.016•
RFR	0.620±0.016•	0.093±0.005•	0.081±0.010•	–	0.560±0.088•	0.076±0.020•	0.074±0.020•

Mean ± standard deviation over 30 seeds. Bold: best mean in the block. •/°: FADO significantly better/worse than that method (paired Wilcoxon test, Holm-corrected, $\alpha = 0.05$). λ: fairness weight, the mean of λ_t over the stream for FADO and a constant for the ARANYANI variants.

B. Can Adaptive Mechanisms Preserve Fairness While Maintaining Utility?

Fairness against the published baselines. In every block of Table III, FADO attains a significantly lower demographic parity than each of ARANYANI, ARF, and RFR ($p < 0.001$); relative to ARANYANI, the reduction ranges from 25% (*x_exceed_skip*) to 33% (*y_swaps*). Equalized odds, which FADO does not optimize directly, is also lower than that of the three in four of the five blocks; the exception is *x_exceed_skip*, where ARANYANI attains 0.047 against FADO's 0.048. The advantage holds after the onset, where FADO has the lowest demographic parity of all methods in the four ADULT scenarios, significantly lower than each of the three ($p < 0.001$), and the lowest equalized odds in three. FADO also limits the increases identified in Section V-A: after the onset of *x_permutations*, its equalized odds rises from 0.047 to 0.064, against 0.115 for ARANYANI, and under *y_prior_skip* its increase is not significant.

Utility. The fairness gain comes at a small utility cost relative to ARANYANI. FADO's F1 is 0.758 against 0.762 on FOLKTABLES and is lower on three of the four ADULT drifts, by at most 0.019 (*y_prior_skip*); on *x_permutations* FADO exceeds ARANYANI (0.612 against 0.609), and RFR attains the highest F1. FADO remains well above ARF in every block. The cost is largest after the onset of *y_prior_skip* (0.621 against 0.667), the drift after which FADO raises its fairness weight the most (to 3.01 on average).

Against the leaked weight. Since FADO's mean λ_t exceeds the published weight of ARANYANI, part of its fairness gain could come from a larger weight rather than from its adaptation. The comparison with ARANYANI with leaked λ, pre-trained like FADO so that both start from the same forest,

TABLE IV
DETECTION OF THE INJECTED DRIFT BY FADO ON ADULT, 30 SEEDS PER SCENARIO

	Either detector		Accuracy detector		Fairness monitor		False alarms per run	
Scenario	Detected	Delay	Detected	Delay	Detected	Delay	Accuracy	Fairness
x_permutations	13/30 (43%)	505	11/30 (37%)	426	2/30 (7%)	943	0.17	0.83
y_swaps	16/30 (53%)	116	16/30 (53%)	116	3/30 (10%)	675	0.13	1.83
y_prior_skip	27/30 (90%)	535	10/30 (33%)	781	17/30 (57%)	390	0.47	1.43
x_exceed_skip	2/30 (7%)	866	0/30 (0%)	–	2/30 (7%)	866	0.17	0.40
All	58/120 (48%)	424	37/120 (31%)	388	24/120 (20%)	511	0.23	1.12

Detected: an alarm within 1,000 samples after the onset. Delay: mean number of samples from the onset to that alarm, over the detected runs.

separates the two effects, under conditions that favor the baseline: it is given the weight that FADO found on that very dataset. Under *x_permutations*, FADO outperforms it after the onset on every metric: demographic parity 0.062 against 0.077, equalized odds 0.064 against 0.100, and F1 0.550 against 0.493. A constant weight applies more pressure than needed before the drift and less than needed after it, which an adaptive weight avoids: FADO's weight averages 0.97 before the onset and 2.30 after it, against the constant 1.5. Under the other drifts the two are close and trade metrics: the leaked weight attains a lower whole-stream demographic parity under *y_swaps* (0.044 against 0.048) and *x_exceed_skip* (0.053 against 0.057), while FADO attains a lower equalized odds under *y_swaps* and *y_prior_skip*, and a lower demographic parity after the onset of *y_prior_skip* (0.063 against 0.079) at a lower F1. On the gradual shift of FOLKTABLES, the leaked weight is slightly fairer (0.039 against 0.042 in demographic parity) at the same F1. Adapting the weight online thus matches a fixed weight that could only have been obtained by tuning on each dataset, and improves on it under abrupt concept drift.

Drift detection. FADO detects the injected drift within one window in 48% of the ADULT runs (Table IV). The two detectors are complementary: the accuracy detector registers label swaps within about 116 samples, whereas class-prior shifts are mostly registered by the fairness monitor, which raises the detection rate on *y_prior_skip* to 90%. The covariate shift of *x_exceed_skip*, which barely affects the error rate, is detected in only 7% of the runs, and the fairness monitor raises 1.12 unmatched alarms per run, each of which resets the penalty's running statistics. The fairness gain does not depend on detection: in the 62 runs where neither detector fired, FADO's demographic parity after the onset remains lower than ARANYANI's in 61, although by less than in detected runs (-0.013 against -0.035), while its equalized-odds advantage largely requires a detection. The per-scenario split is provided with the code (Section IV-I).

C. Ablation of the Adaptive Constraint

To isolate the adaptive fairness constraint, we ran on FOLKTABLES, on the same 30 seeds as Table III, a variant of FADO that keeps the drift detector and the reaction controller but uses the fixed weight $\lambda = \lambda_{\text{base}}$, without the fairness monitor and the reset. Removing these components worsens every metric on at least 28 of the 30 seeds ($p < 10^{-8}$): demographic parity rises from 0.042 to 0.050, equalized odds from 0.025 to 0.035, and F1 falls from 0.758 to 0.748. The adaptive constraint therefore improves fairness and utility together on this dataset. The gain is concentrated in the first half of the test stream (2017); on the 2018 half the two variants reach a similar demographic parity. A per-component ablation on ADULT is left for future work.

VI. CONCLUSION AND FUTURE WORK

This paper studied group fairness in online learning under concept drift and proposed FADO, which turns the fixed fairness weight of a fair oblique forest into the dual price of a demographic-parity bound, tracked online from an unbiased per-sample estimate of each group's disparity, and couples it with drift detection, a fairness monitor, and a reaction controller.

Regarding RQ1, concept drift does not degrade fairness uniformly. On ADULT, the drifts that change the relation between inputs and labels widened the error-rate gap of the fixed-weight learner, whose equalized odds rose from 0.047 to 0.115 after the feature permutation, whereas relabeling one class or removing part of the input space narrowed its demographic parity. A fairness level calibrated before a drift is therefore not guaranteed after it, and which criterion deteriorates depends on the drift. Regarding RQ2, FADO lowered demographic parity by 25 to 33% relative to its base learner in every block and was significantly fairer in demographic parity than each of the three published baselines in all 27 comparisons, over the whole stream and after the onset, at a whole-stream F1 cost of at most 0.019 relative to its base learner. Against the base learner given, in hindsight, the weight that FADO reaches online, it was significantly better in 12 and worse in 7 of 27 comparisons, and better on every metric after the feature permutation, where a constant weight is too small once the drift begins. On FOLKTABLES, removing the adaptive constraint worsened fairness and F1 together.

FADO has limitations that point to future work. Its detectors registered 48% of the injected onsets within one window and only 7% of the covariate shifts, which calls for detectors on fairness-relevant statistics of the input distribution. Finally, our study uses one binary protected attribute, two income datasets, and abrupt injected drifts; more groups and intersectional attributes, gradual and recurring drift, a per-component ablation on ADULT, and a runtime analysis are left for future work.

Acknowledgment

The authors used generative AI, to help edit parts of the text, help generate figures and brainstorm on tables' format. All content was reviewed and any errors or inaccuracies are the authors' own.

References

[1] K. Shailaja, B. Seetharamulu, and M. A. Jabbar, "Machine learning in healthcare: A review," in *Proc. 2nd Int. Conf. Electron., Commun. Aerosp. Technol. (ICECA)*, 2018, pp. 910–914.

[2] N. T. Singh, M. Mehra, I. Verma, N. Singh, D. Gandhi, and M. A. Alladin, "Advancing crime analysis and prediction: A comprehensive exploration of machine learning applications in criminal justice," in *Proc. 2nd Int. Conf. Intell. Data Commun. Technol. Internet Things (IDCIoT)*, 2024, pp. 1339–1343.

[3] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, "Mitigating bias in algorithmic hiring: Evaluating claims and practices," in *Proc. Conf. Fairness, Accountability, Transparency (FAT*)*, 2020, pp. 469–481.

[4] N. Mehrabi, F. Morstatter, N. Saxena, K. Lerman, and A. Galstyan, "A survey on bias and fairness in machine learning," *ACM Comput. Surv.*, vol. 54, no. 6, pp. 115:1–115:35, Jul. 2021.

[5] C. Dwork, M. Hardt, T. Pitassi, O. Reingold, and R. Zemel, "Fairness through awareness," in *Proc. 3rd Innov. Theor. Comput. Sci. Conf. (ITCS)*, 2012, pp. 214–226.

[6] M. Hardt, E. Price, and N. Srebro, "Equality of opportunity in supervised learning," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 29, 2016.

[7] G. I. Webb *et al.*, "Characterizing concept drift," *CoRR*, vol. abs/1511.03816, 2015. [Online]. Available: http://arxiv.org/abs/1511.03816

[8] H. M. Gomes, A. Bifet *et al.*, "Adaptive random forests for evolving data stream classification," *Mach. Learn.*, vol. 106, no. 9–10, p. 1469–1495, Oct. 2017. [Online]. Available: https://doi.org/10.1007/s10994-017-5642-8

[9] M. Klaiber, M. Rössle, and A. Theissler, "The 10 most popular concept drift algorithms: An overview and optimization potentials," *Procedia Computer Science*, vol. 225, pp. 1261–1271, 2023, 27th International Conference on Knowledge Based and Intelligent Information and Engineering Sytems (KES 2023). [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1877050923012723

[10] O. B. Deho, M. Bewong, S. Kwashie, J. Li, J. Liu, L. Liu, and S. Joksimovic, "Is it still fair? a comparative evaluation of fairness algorithms through the lens of covariate drift," 2024. [Online]. Available: https://arxiv.org/abs/2409.12428

[11] S. B. R. Chowdhury *et al.*, "Enhancing group fairness in online settings using oblique decision forests," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=E1NxN5QMOE

[12] V. Cerqueira, H. M. Gomes, M. Heyden, B. Pfahringer, and A. Bifet, "A framework for evaluating and benchmarking concept drift detection methods," in *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, ser. KDD '26. New York, NY, USA: Association for Computing Machinery, 2026, p. 8671–8682. [Online]. Available: https://doi.org/10.1145/3770855.3819070

[13] Z. Jiang *et al.*, "Chasing fairness under distribution shift: A model weight perturbation approach," 2023. [Online]. Available: https://arxiv.org/abs/2303.03300

[14] T. Salazar, J. Gama, H. Araújo, and P. H. Abreu, "Unveiling group-specific distributed concept drift: A fairness imperative in federated learning," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 37, no. 1, p. 161–175, Jan. 2026. [Online]. Available: http://dx.doi.org/10.1109/TNNLS.2025.3601834

[15] Z. Xiong, N. Dalmasso, A. Mishler, V. K. Potluru, T. Balch, and M. Veloso, "FairWASP: Fast and optimal fair Wasserstein pre-processing," in *Proc. AAAI Conf. Artif. Intell.*, vol. 38, no. 14, Mar. 2024, pp. 16 120–16 128.

[16] X. Zeng, J. Ward, and G. Cheng, "FairRR: Pre-processing for group fairness through randomized response," arXiv:2403.07780, 2024. [Online]. Available: https://arxiv.org/abs/2403.07780

[17] M. B. Zafar, I. Valera, M. G. Rodriguez, and K. P. Gummadi, "Fairness constraints: Mechanisms for fair classification," 2017. [Online]. Available: https://arxiv.org/abs/1507.05259

[18] A. Agarwal, A. Beygelzimer, M. Dudík, J. Langford, and H. Wallach, "A reductions approach to fair classification," 2018. [Online]. Available: https://arxiv.org/abs/1803.02453

[19] A. Cotter, H. Jiang, S. Wang, T. Narayan, M. Gupta, S. You, and K. Sridharan, "Optimization with non-differentiable constraints with applications to fairness, recall, churn, and other goals," 2018. [Online]. Available: https://arxiv.org/abs/1809.04198

[20] M. Mahdavi, R. Jin, and T. Yang, "Trading regret for efficiency: Online convex optimization with long term constraints," *J. Mach. Learn. Res.*, vol. 13, pp. 2503–2528, 2012.

[21] A. Mishler, E. H. Kennedy, and A. Chouldechova, "Fairness in risk assessment instruments: Post-processing to achieve counterfactual equalized odds," in *Proc. ACM Conf. Fairness, Accountability, Transparency (FAccT)*, Mar. 2021, pp. 386–400. [Online]. Available: http://dx.doi.org/10.1145/3442188.3445902

[22] E. Small, K. Sokol, D. Manning, F. D. Salim, and J. Chan, "Equalised odds is not equal individual odds: Post-processing for group and individual fairness," in *Proc. ACM Conf. Fairness, Accountability, Transparency (FAccT)*, Jun. 2024, pp. 1559–1578. [Online]. Available: http://dx.doi.org/10.1145/3630106.3658989

[23] W. Zhang and E. Ntoutsi, "Faht: An adaptive fairness-aware decision tree classifier," 2019. [Online]. Available: https://arxiv.org/abs/1907.07237

[24] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.

[25] D. G. Horvitz and D. J. Thompson, "A generalization of sampling without replacement from a finite universe," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 663–685, 1952. [Online]. Available: http://www.jstor.org/stable/2280784

[26] A. Bifet and R. Gavaldà, "Learning from time-changing data with adaptive windowing," in *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*. SIAM, 2007, pp. 443–448. [Online]. Available: https://epubs.siam.org/doi/abs/10.1137/1.9781611972771.42

[27] F. Ding, M. Hardt, J. Miller, and L. Schmidt, "Retiring adult: New datasets for fair machine learning," 2022. [Online]. Available: https://arxiv.org/abs/2108.04884

[28] B. Becker and R. Kohavi, "Adult," UCI Machine Learning Repository, 1996, DOI: https://doi.org/10.24432/C5XW20.

[29] J. Montiel, M. Halford, S. M. Mastelini, G. Bolmier, R. Sourty, R. Vaysse, A. Zouitine, H. M. Gomes, J. Read, T. Abdessalem, and A. Bifet, "River: machine learning for streaming data in python," 2020. [Online]. Available: https://arxiv.org/abs/2012.04740

[30] J. Gama, R. Sebastião, and P. P. Rodrigues, "On evaluating stream learning algorithms," *Mach. Learn.*, vol. 90, no. 3, p. 317–346, Mar. 2013. [Online]. Available: https://doi.org/10.1007/s10994-012-5320-9

[31] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.